AskDocs AI

SYNOPSIS

Project Phase II

MASTER OF COMPUTER APPLICATIONS

4TH SEMESTER

Submitted by

Kshitij Gupta
Batch Year : 2024-2026
Enrolment No: U2449028

PROJECT GUIDE – Ms. Komal Singh

Centre of Computer Education & Training
Institute of Professional Studies
University of Allahabad, Prayagraj
Uttar Pradesh - 211002

Table of Content

Sr. No.	Contents	Page No.
1.	Introduction	iii-v
	1.1. Problem Definition	iv
	1.2. Motivations	v
2.	Objective	vi
3.	Requirement Analysis And Specification	vii-x
	3.1 Functional Requirements	vii
	3.2 Non-Functional Requirements	viii-ix
	3.3 Software & Hardware Requirements	x
4.	System Design DFD (Data Flow Diagram)	xi
5.	Module Design	xii-xiii
6.	Milestones	xiv
7.	Meetings with Supervisor	xv
8.	References	xvi

1. INTRODUCTION

Project Name: AskDocs AI – AI-Powered Document Intelligence

Category: AI-Powered Web Application (Semantic Q&A System)

AskDocs AI is a web-based intelligent document question-answering system that allows users to upload documents such as PDFs, text files, and reports, and then ask questions in natural language to retrieve precise, context-aware answers. Instead of relying on traditional keyword-based search, the system uses semantic search with vector embeddings to understand the meaning of both the stored documents and user queries, returning the most relevant passages along with transparent source references. This project showcases the integration of Natural Language Processing (NLP), vector databases, and backend APIs using Django and Django REST Framework, targeting real-world use cases such as academic note search, enterprise documentation lookup, research paper analysis, and legal/policy document querying. The system workflow includes document ingestion, text preprocessing, embedding generation, semantic retrieval, and answer generation with traceable sources.

1.2. PROBLEM DEFINITION

Organizations, students, and researchers increasingly work with large, unstructured collections of documents, making manual search slow, error-prone, and inefficient. Traditional search tools largely depend on exact keyword matching, which often fails when users phrase queries differently from the text in the documents or when the relevant information is spread across multiple sections. There is a clear gap for a system that can understand semantic similarity, retrieve contextually relevant text spans, and present concise answers instead of just listing documents.

Many existing solutions either offer simple full-text search or require complex configuration and are not easily extensible for small academic or institutional use. Users also struggle with traceability of answers, as systems rarely show the exact source segments backing a response, reducing trust in the results. AskDocs AI addresses these challenges by combining semantic embeddings, similarity search, and an explainable Q&A interface that always links back to the underlying documents.

Thus, there is a need for a lightweight, explainable, and easily deployable semantic document intelligence system suitable for academic and institutional environments.

Key Challenges

1. **Inefficient Keyword Search:** Conventional search fails when users do not know the exact terms used in the documents, leading to missed or irrelevant results.
2. **Scattered and Unstructured Content:** Important information is buried across long PDFs, technical reports, and policy documents, making manual navigation time-consuming.
3. **Lack of Explainability:** Many AI tools do not clearly show which document sections support a given answer, reducing user confidence in the system's output.
4. **Scalability of Manual Search:** As the number and size of documents grow, manual reading and searching becomes practically impossible for timely decision-making.
5. **Access Control and Security:** Sensitive internal documents require authenticated access, role-based permissions, and secure APIs for safe usage.
6. **Integration Complexity:** Building a system that connects document ingestion, text extraction, embedding generation, vector indexing, and Q&A logic requires careful backend design and modular architecture.

1.3. MOTIVATION

Students, teachers, and professionals often maintain large collections of notes, project reports, and reference PDFs but struggle to quickly find exact answers to specific questions across these documents. An intelligent system that can understand questions in natural language and return concise, source-backed answers would significantly improve productivity and learning effectiveness. For example, a student could query a semester's worth of notes, or an organization could search internal policies without manually browsing each file.

The motivation behind AskDocs AI is to build a practical, real-world application that bridges the gap between static document storage and interactive knowledge access. By integrating Django, REST APIs, semantic embeddings, and a vector database, the project also provides an excellent opportunity to gain hands-on experience with modern AI-driven backend systems. Additionally, the project emphasizes explainability and security, which are essential for academic and enterprise adoption.

2. OBJECTIVE

This project aims to develop a secure, scalable, AI-powered web application that converts uploaded documents into a searchable semantic knowledge base and allows users to ask natural language questions to retrieve accurate, context-aware answers with clear source references.

Goals

1. Design and implement a Django-based backend that supports user authentication, role-based access (Admin/User), and RESTful APIs for document upload and question-answering.
2. Build a robust document ingestion pipeline to handle PDF and text uploads, perform text extraction, cleaning, and chunking into manageable sections.
3. Generate vector embeddings for document chunks using sentence-transformer models and store them in a FAISS-based vector database for efficient similarity search.
4. Implement a semantic search and Q&A engine that embeds user queries, retrieves top relevant chunks, and constructs coherent answers enriched with source metadata.
5. Ensure secure handling of user data and documents through JWT-based authentication, validation, and controlled access to APIs.
6. Provide a user-friendly and performant interface (or API layer) that supports pagination, error handling, and extensibility for future front-end or enterprise integration.

3. REQUIREMENT ANALYSIS AND SPECIFICATION

3.1 Functional Requirements

1. **User Registration & Authentication**

   - Allow users to register and log in using email and password.
   - Implement secure authentication using JWT tokens integrated with Django REST Framework.
   - Support role-based access (Admin, Standard User) to protect administrative endpoints.

2. **User Profile Management**

   - Store basic user details, roles, and activity logs in the relational database.
   - Allow admins to view user lists, uploaded documents, and usage statistics.

3. **Document Upload & Management**

   - Provide secure endpoints for uploading documents (PDF, text files).
   - Store original files and associated metadata (title, description, owner, upload date).
   - Support listing, viewing metadata, and soft-deleting or archiving documents.

4. **Text Extraction & Preprocessing**

   - Extract text from PDFs using libraries like pdfplumber or PyPDF2.
   - Clean extracted text by removing extra whitespace, headers, and unnecessary artifacts.
   - Split text into chunks of configurable size and assign unique chunk IDs.

5. **Embedding Generation & Vector Storage**

   - Use sentence-transformers to convert each text chunk into a dense vector embedding.
   - Store embeddings in a FAISS index, linked to chunk IDs and document metadata in PostgreSQL.
   - Provide utilities to rebuild or update the index when new documents are added.

6. **Semantic Search & Question Answering**

   - Embed user questions using the same sentence-transformer model.
   - Perform similarity search over the FAISS index to retrieve top-k relevant chunks.
   - Build a Q&A pipeline that combines retrieved chunks and generates a final structured answer with references

7. **API Endpoints & Response Handling**

   - Implement REST APIs for document upload, listing documents, asking questions, and getting answers with source references.
   - Validate inputs, handle common errors gracefully, and return consistent JSON responses.

3.2. Non Functional Requirements

1. **Performance:**
    - The system should respond to typical Q&A requests within 2–5 seconds for moderate document collections.
    - Embedding and indexing operations should be batched to optimize throughput.
2. **Scalability:**
    - The architecture should support growth from a few hundred to several thousand documents without major redesign.
    - FAISS index and PostgreSQL database should be configurable to run on higher-capacity servers if needed.
3. **Security:**
    - All protected endpoints must require valid JWT tokens.
    - Implement role-based access control for admin operations and document visibility.
    - Follow secure coding practices to prevent common vulnerabilities such as injection and insecure direct object references.
4. **Reliability & Maintainability:**
    - Use Django's ORM and migrations to maintain database schema reliably.
    - Structure the codebase into modular Django apps (users, documents, qa_engine) to simplify maintenance and testing.
5. **Usability:**
    - Offer clear API documentation and meaningful error messages for frontend or API consumers.
    - Ensure that the question-answering flow is intuitive and that answers always include identifiable sources.
6. **Avalibality:**
    - The system should handle partial failures (e.g., vector index reload) without crashing the entire application.

Technical Requirements

- **Programming Languages:**
  - **Python 3** (for backend development and AI processing).
- **Frameworks:**
  - **Backend (Web API):** Django framework with Django REST Framework (DRF) for secure RESTful APIs.
  - **API Development:** Django REST Framework for JWT authentication, serialization, and endpoint management.
- **Database:**
  - **PostgreSQL** to securely store user profiles, document metadata, chunk information, and query logs.
  - **Vector Database:** FAISS (Facebook AI Similarity Search) for storing and querying document embeddings
- **Authentication & Security:**
    - JWT (JSON Web Tokens) for stateless user authentication and authorization.
    - Role-based access control (Admin/User roles) with Django permissions.
- **NLP & AI Libraries:**
  - **Sentence-Transformers** for generating semantic embeddings from text chunks and user queries.
  - **pdfplumber/PyPDF2** for PDF text extraction, table parsing, and metadata retrieval.
- **Document Processing & Modules:**
  - **Text Extraction & Chunking:** Automated pipeline for PDF parsing, text cleaning, and intelligent chunking (300-500 words per chunk).
  - **Embedding Pipeline:** Batch processing to convert document chunks into dense vector representations.
- **Containerization & Deployment:**
  - **Docker** for containerizing Django application, PostgreSQL, and FAISS services.
  - **Environment Variables** for secure configuration management (API keys, database credentials).
- **Development & Testing Tools:**
  - **Development IDE:** VS Code / PyCharm with Python and Django extensions.
  - **API Testing:** Postman / Insomnia for testing REST endpoints and JWT authentication.
  - **Version Control:** Git + GitHub for source code management and collaboration.
  - **Unit Testing:** Django's TestCase framework with pytest for backend testing.

3.3. Software & Hardware Requirements

1. Hardware Requirements

Category	Minimum Requirement	Recommended Requirement
Processor	Intel i3 (10th Gen) / AMD equivalent	Intel i5/i7 (11th Gen+) / Apple M1+
RAM	4 GB	8 GB or higher
Storage	20 GB free space	50 GB SSD or higher
Internet	5 Mbps connection	20 Mbps or higher

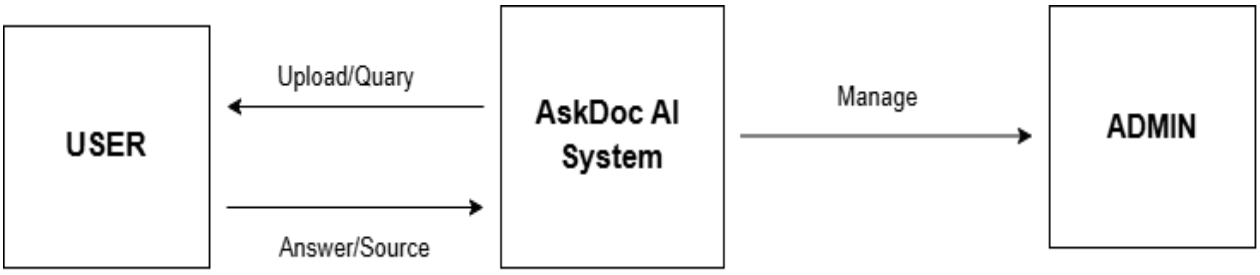
2. Software Requirements

Category	Specification
Operating System	Win Windows 10/11, macOS, or Ubuntu Linux
Programming Language	Python 3.10+
Web Frameworks	Django 4.x + Django REST Framework
Database	PostgreSQL 14+
Authentication	Django rest framework- simple jwt (JWT)
Containerization	Docker & Docker Compose
IDE	VS Code
Version Control	Git + GitHub

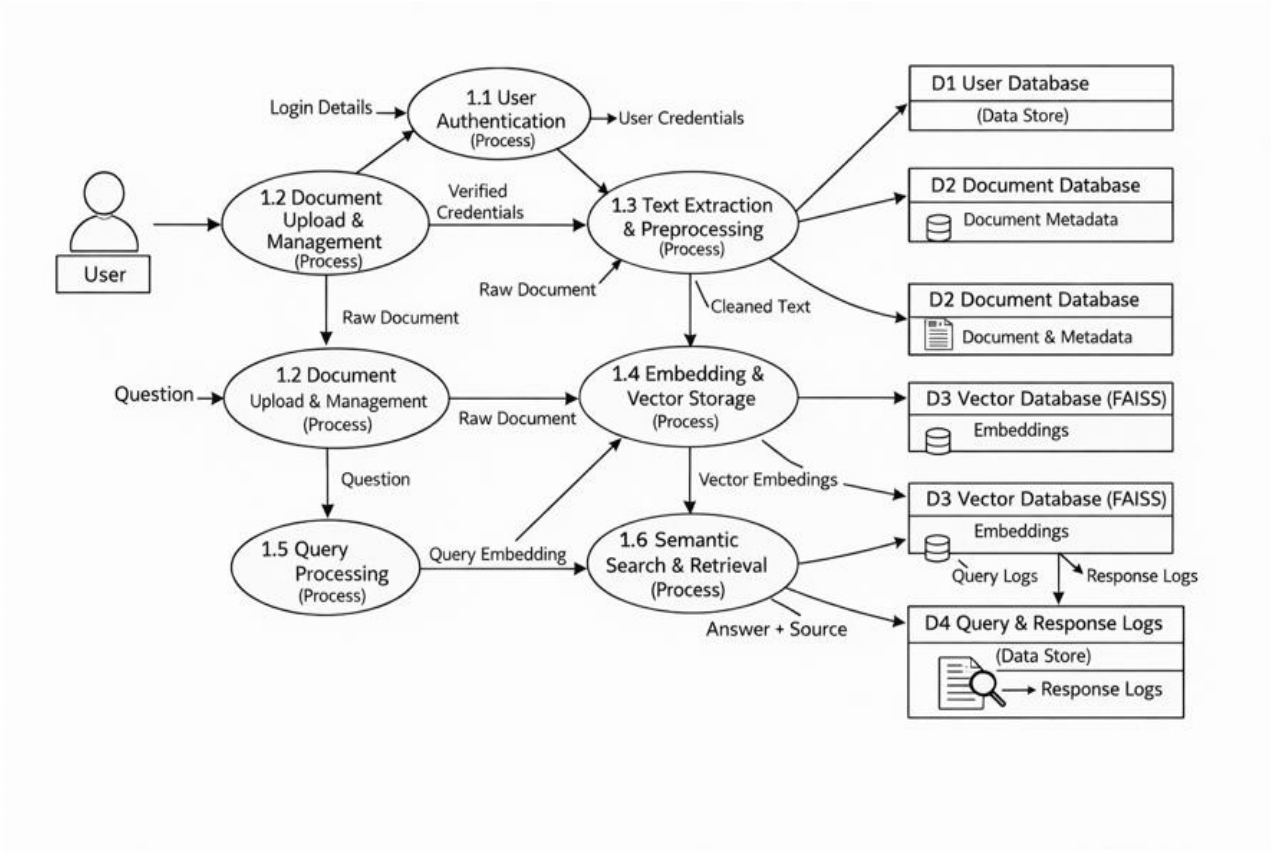
4. SYSTEM DESIGN

System Design of AskDocs AI Using the Data Flow Diagram **(DFD)**

Level 0:



Level 1:



5. MODULES DESIGN

- **User Authentication Module**
  - **Type**: JWT-based Token Authentication
  - **User submits credentials** → Django validates against hashed password in PostgreSQL → Issues access and refresh tokens.
  - Role check middleware protects Admin and User-specific endpoints
- **Document Management & Storage Module**
  - Type: File Upload + CRUD Operations
  - User uploads PDF/text via POST /api/documents/upload/.
  - File is saved to media storage; metadata stored in Document model in PostgreSQL.
  - pdfplumber extracts raw text page-by-page; text stored in Extracted Text model.
- **Text Processing & Chunking Module**
  - Algorithm: Sliding Window Chunking with Overlap
  - Extracted text is cleaned (normalize whitespace, remove special characters).
  - Text is split into chunks of 300–500 words with 50-word overlap to preserve contextual boundaries.
  - Each chunk is stored with its document ID and position index.
- **Embedding & FAISS Indexing Module**
  - Algorithm: Semantic Embedding using all-MiniLM-L6-v2 (Sentence Transformers)
  - Each chunk is passed through the embedding model to produce a 384-dimensional vector.
  - Vectors are stored in a FAISS IndexFlatL2 or IndexIVFFlat index for efficient retrieval.
  - Index is saved to disk and reloaded on system restart.
- **Question Answering & Search Module**
  - Algorithm: Cosine Similarity / L2 Distance Search in FAISS
  - User question is embedded using the same Sentence Transformer model.
  - FAISS performs k-NN search and returns top-5 most semantically similar chunks.
  - Retrieved chunks are assembled as a context window.
  - Depending on configuration, answers may be extractive (span-based) or generative (LLM-assisted).

- **API Layer Module (Django REST Framework)**
  - POST /api/auth/register/ – User registration
  - POST /api/auth/login/ – JWT login
  - POST /api/documents/upload/ – Upload document
  - GET /api/documents/ – List user documents
  - POST /api/qa/ask/ – Submit a question and get an AI answer

6. MILESTONES

Sr. No.	Project Activity	Estimated Start Date	Estimated End Date
1.	Project Allotment	05/02/2026	12/02/2026
2.	Synopsis Creation	13/02/2026	24/02/2026
3.	Project Synopsis Presentation	25/02/2026	15/03/2026
4.	Frontend Development	16/03/2026	30/03/2026
5.	Back end Development	31/03/2026	15/04/2026
6.	Integration and Testing	16/04/2026	30/04/2026
7.	Documentation	01/05/2026	10/05/2026
8.	Final Presentation	11/05/2026	20/05/2026

7. MEETINGS WITH SUPERVISOR

Date of the Meeting	Mode	Comments by the Supervisor	Signature of the Supervisor

8. References

- **Reimers, N., & Gurevych, I. (2019).** Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. arXiv:1908.10084. https://arxiv.org/abs/1908.10084
- **Johnson, J., Douze, M., & Jégou, H. (2017)**. Billion-scale similarity search with GPUs. arXiv:1702.08734. https://arxiv.org/abs/1702.08734
- **Django Software Foundation. (2024)**. Django Documentation. https://docs.djangoproject.com
- **Hugging Face. (2024)**. Sentence Transformers Documentation. https://www.sbert.net
- **Meta AI Research. (2024)**. FAISS – Facebook AI Similarity Search. https://faiss.ai
- **pdfplumber. (2024)**. GitHub Repository. https://github.com/jsvine/pdfplumber
- **Simple JWT. (2024)**. Django REST Framework SimpleJWT Documentation. https://django-rest-framework-simplejwt.readthedocs.io
- **PostgreSQL Global Development Group. (2024)**. PostgreSQL Documentation. https://www.postgresql.org/docs